

OctoNihon #7

定める。測る。揃える。任せる。



AKITAKA SHINBA
Solutions Engineer
GitHub



Akitaka Shinba

SOLUTIONS ENGINEER / GITHUB

GitHub Solutions Engineer。ソフトウェアエンジニアとしての開発経験、開発チームへの技術活用支援や人材育成の経験を活かし、現在はGitHub EnterpriseやGitHub Copilotを中心に、お客様の開発者体験の向上やAIを活用したソフトウェア開発を支援している。

GitHub入社以前にはGitHub Starとして活動。現在も、技術トレーニングやワークショップの提供、コミュニティでの登壇や情報発信に取り組んでいる。

趣味： 愛犬とおでかけ 🐕



定める。測る。揃える。任せる。

この夏、GitHub Enterprise の管理機能に入ったアップデートを、
この4つの視点でご紹介します。



Session Agenda

- 01 定める ルールを、rulesets で定め直す
- 02 測る 効いているかを測り、証拠を残す
- 03 揃える 標準を、全リポジトリで揃える
- 04 任せる AIに、どこまで任せるか
- 05 明日からやることリスト



01 定める

ルールを、rulesets で定め直す



リポジトリごとのブランチ保護から、Org 全体の rulesets へ変換

	Branch protection	Repository rulesets
特定ブランチの保護	✓	✓
同一ブランチに複数ポリシー	—	✓
バイパス管理	✓	✓
開発者がルールを確認	—	✓
Org 全体への適用	—	✓
Push 自体のファイル制約	—	✓
ポリシーのレイヤー	—	✓

Settings → Branches の既存ルールに「Convert to ruleset」ボタン。



Rulesetsが、ブランチ保護を超えた統制基盤になった

統制ポイント	ルール		日付・状態	前提
● push	Push rules のパス例外 ルールは全体に効かせたまま、指定したパスだけ除外	拡張	2026/8/25 Public Preview	—
● CI を起動	Actions ワークフローの実行制御 誰が・どのイベントで CI を起動できるかを定義。evaluate mode で影評価してから施行	新規	2026/9/17 GA	—
● 依存関係	OSS ライセンスコンプライアンス 企業横断のライセンスポリシーを merge 条件に	新規	2026/6/30 Public Preview	GHAS Code Security
● 品質	コードカバレッジによる merge 保護 既定ブランチからの低下幅で merge をブロック	新規	2026/6/30 Public Preview	Code Quality
● レビュー	レビューを却下（Dismiss）の制限 Branch protection にあった機能が rulesets にも揃った	拡張	2026/7/7 GA	—
● merge	秘密情報が未解決の PR をブロック 検知したがブロックしなかった分を、マージ前の 2 つ目のゲートで止める	新規	2026/9/9 Public Preview	Secret Protection
— は追加製品なし（rulesets 標準）				

ブランチ保護から、CI の起動・依存関係のライセンス・シークレット・カバレッジまで。
統制の仕組みが、Rulesetsという共通基盤に集まり始めた。

例外を、申請ではなく 設定に書く

Push rules に「Allowed exceptions」が追加。

ルールは全体に効かせたまま、指定したパス
だけ除外できる。

対象は2ルール：Restrict file paths と Restrict file
size。

これまで：ルールを全員に緩める／
bypass 権限を配る／GitHub の外で守る



これから：ルールはそのまま、パスだけ除外。
bypass を増やさない

Restrict file paths

Restricted: `**/*.jar`

Allowed exceptions: `**/gradle/wrapper/*.jar`

→ .jar は全面禁止。Gradle wrapper だけ許可

Restrict file size

Maximum: 20 MiB

Allowed exceptions: `assets/video/**` (例)

→ 上限を入れつつ、既に超えている既存パスだけ除外。

ファイルを直さずに制限を導入できる

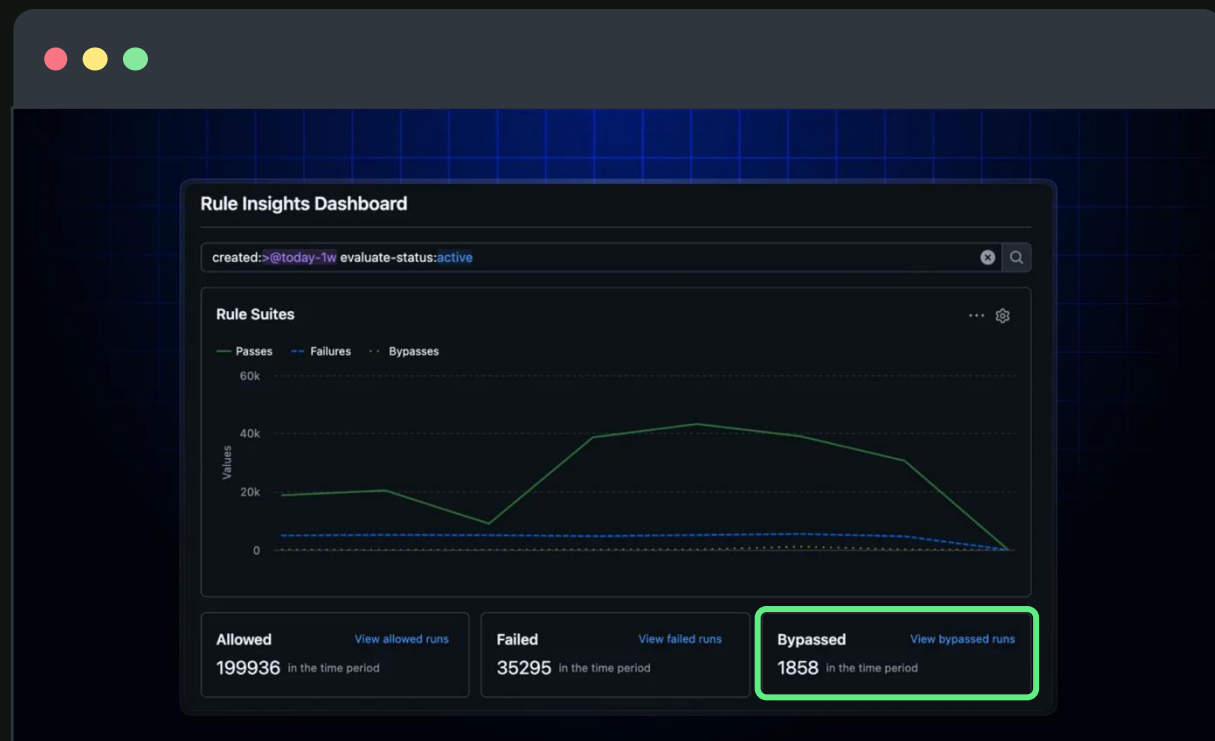
02 測る

効いているかを測り、証拠を残す



「設定した」は「効いている」ではない

先月、誰が何回バイパスしましたか？



3つの数字の読み方

199,936**Allowed**

ルールを満たして通った。

35,295**Failed**

ルールに引っかかって止まった。
多くてもいい。ルールが効いている証拠。

1,858**Bypassed**

誰かが権限で通した。見るのはここ：
誰が・どのルールを・どのリポジトリで。

比率にすると Failed 約 15%、Bypassed 約 0.8%。
でも 1,858 件 — 1 週間なら 1 日 265 件。

「0.8% だから安心」ではなく、
CSV 出しておいて Org 横断でレビューする。

Rulesets の Insights から (Repo: Settings → Rules / Org: Settings → Repository)。バイパス上位・絞り込み・CSV 出力。



GitHub Actions: 実行履歴も Actions Retention に統合

Checks / workflow runs / commit statuses も同じ保持ポリシーで管理

Before — 2026/9/30 まで

Actions Retention (既定 90 日)

Artifacts	90 日
Logs	90 日

Retention 設定の対象外

設定に関係なく保持

Checks	400 日以上
Workflow runs	400 日以上
Commit statuses	400 日以上

2026/10/1



Retention policy を統一

After — 2026/10/1 から

Actions Retention (例 : 90 日)

- Artifacts
- Logs
- Checks
- Workflow runs
- Commit statuses

すべて同じ Retention policy で管理

既存履歴も対象。保持期間を超えた履歴は 10 月 1 日以降削除対象となるため、必要なデータは事前に Export。

「10/1 からさらに 90 日残る」わけではない。削除された履歴は、あとから保持期間を延ばしても復元されない。

90 日は既定値で、Org / Enterprise の設定に従う。

03 揃える

標準を、全リポジトリで揃える

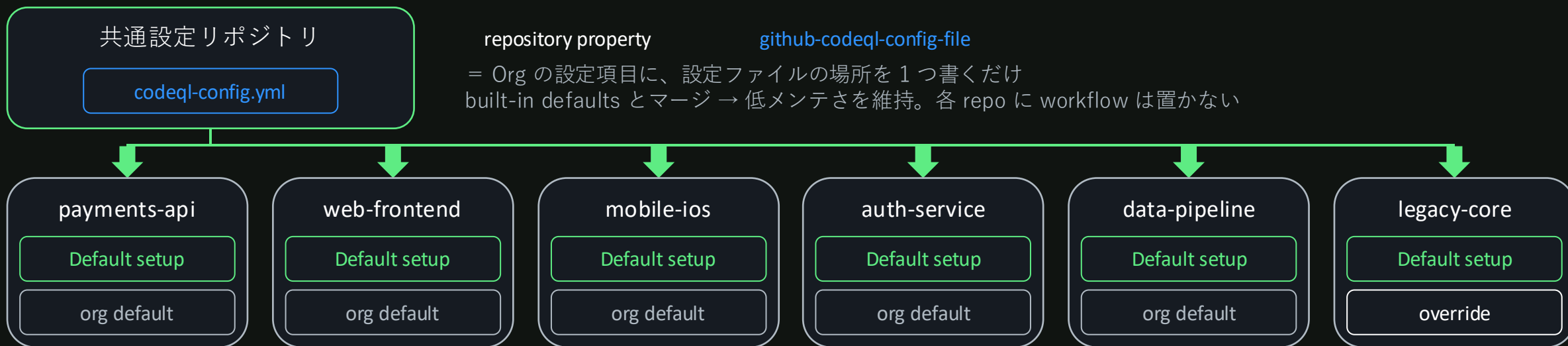


03 揃える

Org内の多くのリポジトリを共通ルールで回す

Code scanning の Default setup に、会社共通の CodeQL 設定を配れるようになった

After 共通設定リポジトリの codeql-config.yml を、repository property で配る



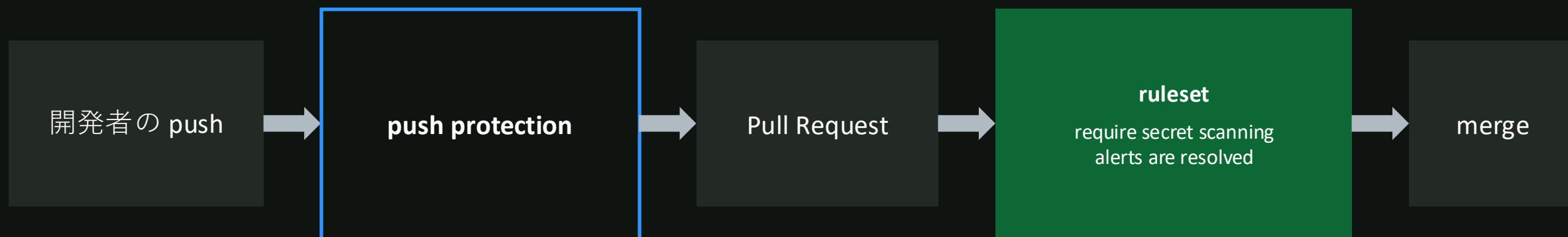
Default setup の楽さと、Advanced setup の粒度の間を埋めるアップデート。

Default setup のまま、会社共通の CodeQL 設定を配れる。

各 repo に workflow を置かずに共通ポリシーを展開でき、標準化と例外対応を両立しやすくなった。

GitHub Code Security (Default setup)。github.com で GA、GHES は 3.23 で提供予定。

セキュリティのルールも、rulesets で決められるようになった



まだブロックされないケース

- 正当な理由により、Push protectionのバイパスが認められた変更
- **Push protectionではブロック対象に設定していないSecret pattern**
例：開発フローへの影響を考慮し、Push時には止めずMerge時に確認するGeneric / Custom pattern
- Push protectionの適用前など、すでにRepositoryへPushされている変更

アラートが未解決の PR はマージ不可

- リポジトリ・組織・エンタープライズのどこからでも設定可
- **Evaluate mode** で、開発現場に影響を与えず試運転可能

Push時に止める Push protection に加えて、**Merge時にも確認する2つ目のゲート**を追加。



04 任せる

AIに、どこまで任せるか



新しいGAモデルは、原則ONへ。

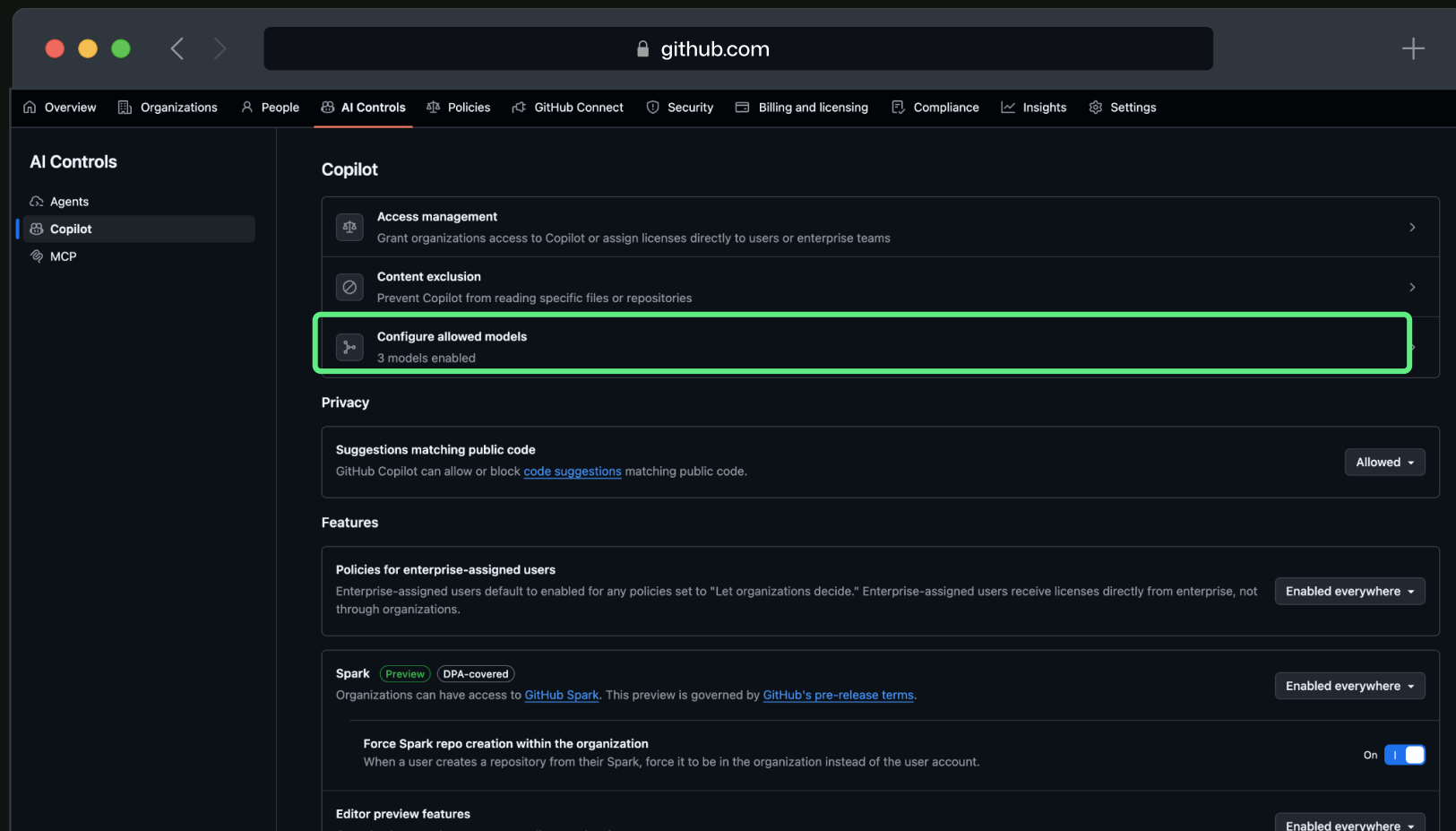
8月26日から順次、明示的に設定していないモデルは「既定ポリシーに従う」状態に

これまで

新しいモデルがGAするたび、管理者が個別にEnable。

これから

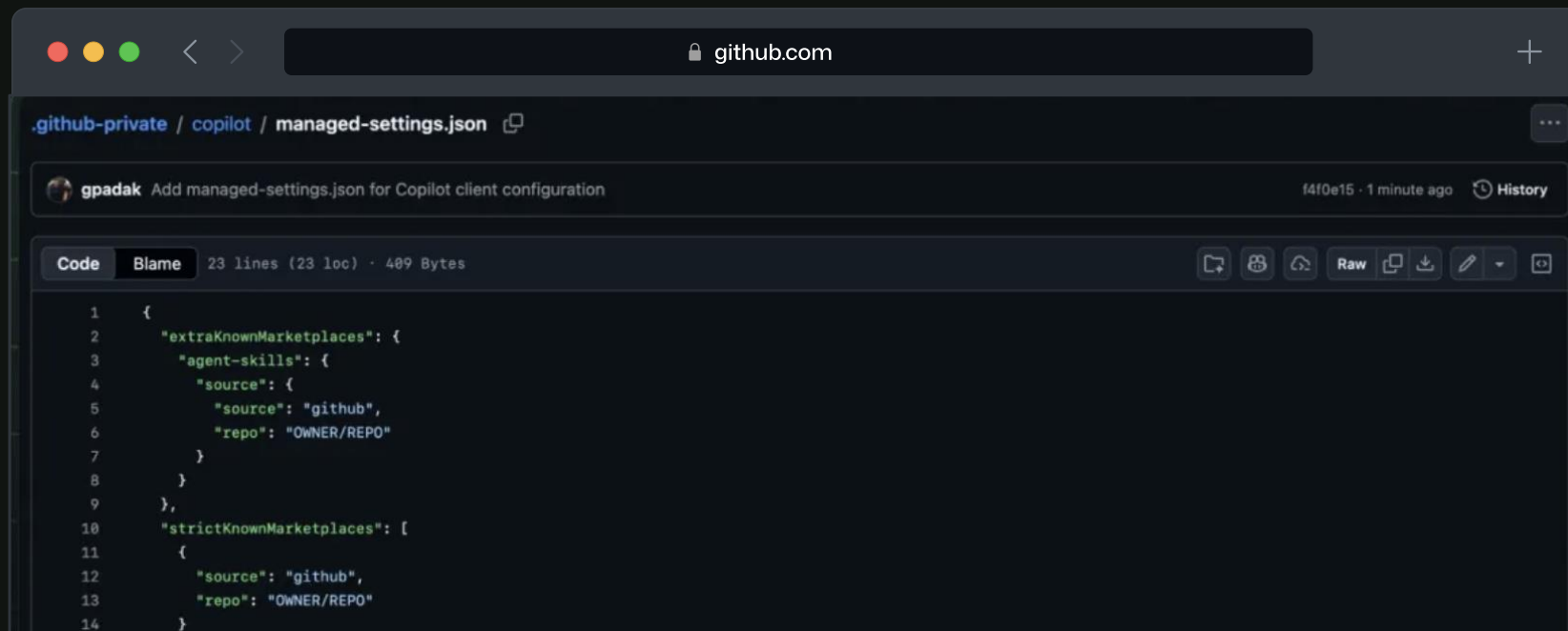
未設定のGAモデルは
Global model policyを自動で継承。
既定値は **Enabled**。



「新モデルは原則ON / 原則OFF」どちらにするか、Default policyを確認。



「全員同じ」から、チームごとの統制へ



```
1 {
2   "extraKnownMarketplaces": {
3     "agent-skills": {
4       "source": {
5         "source": "github",
6         "repo": "OWNER/REPO"
7       }
8     }
9   },
10  "strictKnownMarketplaces": [
11    {
12      "source": "github",
13      "repo": "OWNER/REPO"
14    }
15  ]
16 }
```

.github-private/copilot/managed-settings.json を1か所に置き、エンタープライズチーム別に特化できる。

allowedMcpServers / deniedMcpServers で、Copilot が接続できる MCP サーバーを中央管理。

6/17

yolo モード（自動承認）の禁止



6/25

プラグインの出所を制限



8/6

MCP の allow / deny



9/9

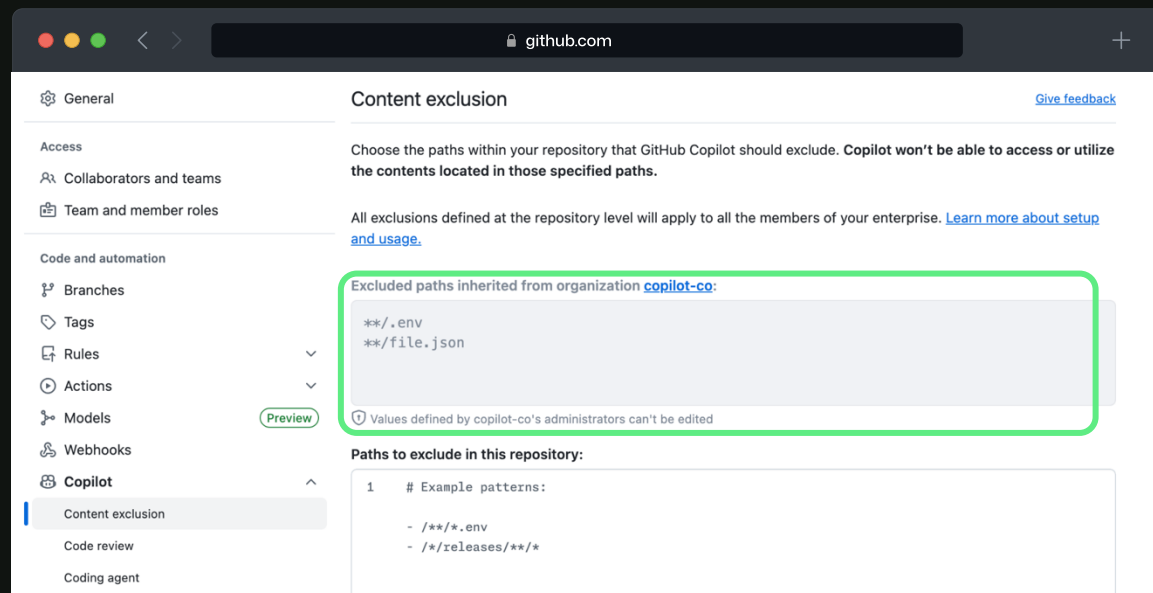
エージェントの操作権限



何を読ませるか。何をさせるか。

読ませる範囲 — Content exclusions

指定したパスは、Copilot が読まない



*対応するクライアント・モードで、指定コンテンツをコンテキストから除外
どちらも管理者が決めたら、開発者側の設定では解除できない（バイパス不可）。

させる操作 — Managed permissions

シェル／ファイル操作／通信先を、ブロック・承認・許可に振り分け

	deny (未許可)	ask (承認制)	allow (許可)
シェルコマンド		✓	
ファイルの読み書き			✓
通信先	✓		

Managed permissions の設定イメージ



Copilot が、Approve できるようになった

Enterprise

「Allow Copilot to approve pull requests」

Disabled everywhere (既定)

Let organizations decide

Enable for selected organizations

Organization

「Count Copilot approvals toward merge requirements」

Enabled everywhere

Let repositories decide

Enable for selected repositories

Disabled everywhere

Repository

「Auto-approval」

Allow Copilot to approve pull requests

File paths (最大 15 glob)

Allow Copilot approvals to count toward merge requirements



Copilot approved these changes

新しいコミットで承認は取り消される

Required approvals 1/1



3 files changed

既定は Disabled everywhere。

「使わせるか」ではなく「どこまで任せるか」を、Enterprise → Organization → Repository の3階層で決める。

Copilot code review は AI credits (と Actions 分) を消費。上限は Budgets で。



統制があるから、広げられる。

定める・測る・揃える・任せる — この4つの道具が、この夏で揃った。



来週やることリスト (copilot まとめ)

★ は期限あり。すべて今日から使える機能。

定める

- Branch protection を rulesets に変換

Repo admin

- 例外を Allowed exceptions に書き、申請運用をやめる

Org owner

- GHES なら 3.22 へ

Enterprise admin

測る

- Rule insights でバイパス上位を見る

Repo admin など・5分

- ★ Actions の保持期間を 10/1 までに見直す

Org / Enterprise owner

- ★ セルフホストランナーを 9/25 までに 2.329.0 以上へ

Platform チーム

揃える

- CodeQL 共通設定を1か所に置く

Security チーム + Org owner

- PR 秘密情報ブロックの ruleset を試す

Org owner

- GHAS トライアル (300 ライセンスまで)

Enterprise owner

任せる

- ★ 今晚 Configure allowed models を確認

Enterprise owner

- managed settings でエージェント権限を「承認要求」から

Enterprise owner

- Copilot approve が Disabled everywhere か確認

Enterprise owner

詳細、設計の相談は GitHub の担当者へ





Thank you

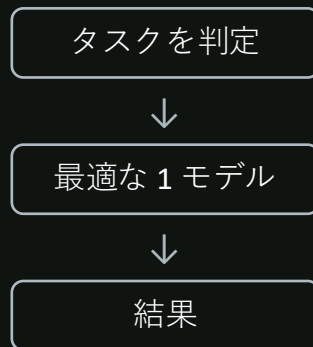
ありがとうございました

最後にひとつ

Auto はモデルを選ぶ。HydraFusion は解き方を選ぶ。

Auto

タスクに最も合う
1つのモデルを選ぶ



モデルを任せる

HydraFusion

タスクに最も合うワークフローを選び、
複数プロバイダーのモデルを組み合わせる

Single

選ばれた 1 つのモデルが直接解く

Cascade

効率的なモデルが下書き → 品質ゲート → 採用、または
より強いモデルへ昇格

Critique

モデル A が下書き → 別ファミリーのモデル B が
読み取り専用でレビュー → モデル A が
1 回修正

解き方を任せる

常に複数モデルを使うわけではない。

品質を満たす最も単純なワークフローを選び、必要なときだけ呼び出しを足す。

開発者は `/model` で HydraFusion を選ぶだけ。

中間の下書きは見せず、1つの結果と1つの変更セットを返す。

最後にひとつ

任せる範囲は、モデルポリシーで決まる

HydraFusion が使えるモデル = 組織が許可したモデル。ルーティングは、その中でだけ行われる。

許可 — モデルポリシー

無効化したモデルはルーティング前に除外。
許可されないモデルを含む計画は実行されない。

使えるモデルが1つも無ければ、選択肢そのものに現れない。

隔離 — レビューと fail-safe

レビュー役はツールを持たない隔離コンテキストで動き、
リポジトリを変更できない。

実行側は通常の権限付きエージェントループ。
キャンセル・検証失敗時はパッチを適用しない。

課金

HydraFusion の追加料金は無い。使ったモデルぶんの合計。

各モデルの標準単価。下書き・批評・修正・昇格・再試行・
フォールバックをすべて集計。安くなる保証はない。

可視性

セッション画面にはモデル名は出ない。

/usage でモデル別の使用量、/collect-debug-logs で
ルーティングの判定と使われたモデルを確認できる。

1 ターンで、複数プロバイダーのモデルにコードが渡り得る。
だから 04 で見た `Configure allowed models` が、ここでも入口になる。

自動レビューが入っても、通常のコードレビュー・テスト・セキュリティ・ガバナンスは変えない。